

# Rapport de TP1 - Introduction à Docker

---

**Étudiants** : Aymane Es-salehy & Nourdine El-Arfaouy  
**Enseignant** : Ahmed Amamou  
**Filière** : Cycle d'Ingénieur Cybersécurité 1ère année  
**Année scolaire** : 2025 - 2026

# Table des matières

|          |                                                    |          |
|----------|----------------------------------------------------|----------|
| <b>1</b> | <b>Introduction</b>                                | <b>2</b> |
| <b>2</b> | <b>Manipulations</b>                               | <b>2</b> |
| 2.1      | Exécution d'un conteneur en arrière-plan . . . . . | 2        |
| 2.2      | Interaction avec un conteneur . . . . .            | 2        |
| 2.3      | Arrêt et suppression . . . . .                     | 2        |
| 2.4      | Déploiement d'un serveur web . . . . .             | 3        |
| 2.5      | Test du serveur . . . . .                          | 3        |
| 2.6      | Observation des logs . . . . .                     | 3        |
| 2.7      | Nettoyage des ressources . . . . .                 | 4        |
| 2.8      | Analyse d'une image Docker . . . . .               | 4        |
| 2.9      | Lancement de l'application . . . . .               | 4        |
| 2.10     | Test de l'API . . . . .                            | 4        |
| 2.11     | Persistance des données . . . . .                  | 4        |
| 2.12     | Inspection du volume . . . . .                     | 5        |
| 2.13     | Volume vs Bind mount . . . . .                     | 5        |
| 2.14     | Nettoyage final . . . . .                          | 6        |
| <b>3</b> | <b>Conclusion</b>                                  | <b>6</b> |

# 1 Introduction

Ce TP a pour objectif de découvrir les bases de Docker : exécution de conteneurs, interaction, déploiement d'applications et gestion des données.

## 2 Manipulations

### 2.1 Exécution d'un conteneur en arrière-plan

```
hodhod@hodhod-VMware-Virtual-Platform:~$ docker run -d --name ping-box alpine:3.20 sh -c "while true; do echo 'je suis vivant'; sleep 2; done"
986f5cc53255ce2ffb56aaed274f9509fb6aa9f1bc182fd7b9b4a651c6c3c5a
hodhod@hodhod-VMware-Virtual-Platform:~$ docker ps
CONTAINER ID   IMAGE          COMMAND                  CREATED        STATUS        PORTS
NAMES
986f5cc53255   alpine:3.20    "sh -c 'while true; ..." 59 seconds ago Up 58 seconds
ping-box
hodhod@hodhod-VMware-Virtual-Platform:~$ docker logs ping-box
je suis vivant
je suis vivant
je suis vivant
je suis vivant
je suis vivant
je suis vivant
```

Un conteneur peut fonctionner en arrière-plan tant que son processus principal est actif.

### 2.2 Interaction avec un conteneur

```
hodhod@hodhod-VMware-Virtual-Platform:~$ docker exec -it ping-box sh
/ # ps aux
PID   USER     TIME   COMMAND
    1   root      0:00   sh -c while true; do echo 'je suis vivant'; sleep 2; done
    65   root      0:00   sh
    72   root      0:00   sleep 2
    73   root      0:00   ps aux
/ # exit
```

Docker permet d'accéder directement au conteneur pour l'inspecter ou le configurer.

### 2.3 Arrêt et suppression

```
hodhod@hodhod-VMware-Virtual-Platform:~$ docker stop ping-box
ping-box
hodhod@hodhod-VMware-Virtual-Platform:~$ docker ps -a
CONTAINER ID   IMAGE          COMMAND                  CREATED        STATUS        PORTS
NAMES
986f5cc53255   alpine:3.20    "sh -c 'while true; ..." 3 minutes ago   Exited (137) 34 seconds
ping-box
c503920d1a8d   alpine:3.20    "sh"                    4 minutes ago   Exited (0) 3 minutes ago
alpine-test
hodhod@hodhod-VMware-Virtual-Platform:~$ docker rm ping-box
ping-box
hodhod@hodhod-VMware-Virtual-Platform:~$ docker rm alpine-test
alpine-test
```

Les conteneurs peuvent être arrêtés puis supprimés pour libérer les ressources.

## 2.4 Déploiement d'un serveur web

```
hodhod@hodhod-VMware-Virtual-Platform:~$ docker run -d --name web -p 8080:80 nginx:1.27-alpine
Unable to find image 'nginx:1.27-alpine' locally
1.27-alpine: Pulling from library/nginx
39c2ddfd6010: Pull complete
81bd8ed7ec67: Pull complete
61ca4f733c80: Pull complete
b464cfd2a63: Pull complete
d7e507024086: Pull complete
34a64644b756: Pull complete
197eb75867ef: Pull complete
f18232174bc9: Pull complete
2a84448aca9c: Download complete
9261b9aff737: Download complete
Digest: sha256:65645c7bb6a0661892a8b03b89d0743208a18dd2f3f17a54ef4b76fb8e2f2a10
Status: Downloaded newer image for nginx:1.27-alpine
51fd7bf813e31982d6a75d69148d8d4aa54cc4575816ef2f7eae9ccb5efe3ee3
```

Un service peut être exposé via un port pour être accessible depuis l'extérieur.

## 2.5 Test du serveur

```
hodhod@hodhod-VMware-Virtual-Platform:~$ curl http://localhost:8080
<!DOCTYPE html>
<html>
<head>
<title>Welcome to nginx!</title>
<style>
html { color-scheme: light dark; }
body { width: 35em; margin: 0 auto;
font-family: Tahoma, Verdana, Arial, sans-serif; }
</style>
</head>
<body>
<h1>Welcome to nginx!</h1>
<p>If you see this page, the nginx web server is successfully installed and
working. Further configuration is required.</p>

hodhod@hodhod-VMware-Virtual-Platform:~$ curl -I http://localhost:8080
HTTP/1.1 200 OK
Server: nginx/1.27.5
Date: Wed, 22 Apr 2026 09:38:26 GMT
Content-Type: text/html
Content-Length: 615
Last-Modified: Wed, 16 Apr 2025 12:55:34 GMT
Connection: keep-alive
ETag: "67ffa8c6-267"
Accept-Ranges: bytes
```

Les requêtes HTTP permettent de vérifier le bon fonctionnement du serveur.

## 2.6 Observation des logs

```
172.17.0.1 - - [22/Apr/2026:09:38:02 +0000] "GET / HTTP/1.1" 200 615 "-" "curl/8.5.0" "-"
172.17.0.1 - - [22/Apr/2026:09:38:26 +0000] "HEAD / HTTP/1.1" 200 0 "-" "curl/8.5.0" "-"
172.17.0.1 - - [22/Apr/2026:09:39:19 +0000] "HEAD / HTTP/1.1" 200 0 "-" "curl/8.5.0" "-"
```

Les logs fournissent des informations en temps réel sur l'activité du conteneur.

## 2.7 Nettoyage des ressources

```
hodhod@hodhod-VMware-Virtual-Platform:~$ docker stop web
web
hodhod@hodhod-VMware-Virtual-Platform:~$ docker rm web
web
```

Le nettoyage permet d'éviter l'accumulation de ressources inutilisées.

## 2.8 Analyse d'une image Docker

```
hodhod@hodhod-VMware-Virtual-Platform:~/devops-monitor$ docker history devops-monitor:1.0
IMAGE          CREATED          CREATED BY          SIZE            COMMENT
3f93e48bc78e   2 minutes ago   CMD ["python" "app.py"]  0B              buildkit.dockerfile.v0
<missing>      2 minutes ago   EXPOSE [5000/tcp]      0B              buildkit.dockerfile.v0
<missing>      2 minutes ago   USER appuser          0B              buildkit.dockerfile.v0
<missing>      2 minutes ago   RUN /bin/sh -c useradd --create-home --shell... 86kB            buildkit.dockerfile.v0
<missing>      2 minutes ago   RUN /bin/sh -c mkdir -p /app/logs # buildkit 12.3kB          buildkit.dockerfile.v0
<missing>      2 minutes ago   COPY app.py . # buildkit 12.3kB          buildkit.dockerfile.v0
<missing>      2 minutes ago   RUN /bin/sh -c pip install --no-cache-dir -r... 18.2MB          buildkit.dockerfile.v0
```

Les images Docker sont construites en couches, ce qui optimise leur utilisation.

## 2.9 Lancement de l'application

```
hodhod@hodhod-VMware-Virtual-Platform:~/devops-monitor$ docker run -d --name monitor -p 8080:5000 devops-monitor:1.0
d491f22f55123b153bc7d31363f309217c88e74b383240e889c22367393b81bd
hodhod@hodhod-VMware-Virtual-Platform:~/devops-monitor$ docker ps
CONTAINER ID   IMAGE          COMMAND                  CREATED        STATUS        PORTS
d491f22f5512   devops-monitor:1.0   "python app.py"         11 seconds ago Up 11 seconds  0.0.0.0:8080->5000/tcp, [::]:8080->5000/tcp   monitor
hodhod@hodhod-VMware-Virtual-Platform:~/devops-monitor$ docker logs monitor
* Serving Flask app 'app'
* Debug mode: off
WARNING: This is a development server. Do not use it in a production deployment. Use a production WSGI server instead.
* Running on all addresses (0.0.0.0)
* Running on http://127.0.0.1:5000
* Running on http://172.17.0.2:5000
Press CTRL+C to quit
```

Une application conteneurisée peut être facilement déployée et exposée.

## 2.10 Test de l'API

```
hodhod@hodhod-VMware-Virtual-Platform:~/devops-monitor$ curl http://localhost:8080/health
{"hostname":"ce408a13cfb6","status":"ok","utc":"2026-04-22T10:31:10.447656Z"}
```

Les endpoints permettent de vérifier l'état de l'application.

## 2.11 Persistance des données

```
hodhod@hodhod-VMware-Virtual-Platform:~/devops-monitor$ docker exec monitor cat /app/logs/visits.log
2026-04-22T10:39:41.738285 /
2026-04-22T10:39:42.754818 /
2026-04-22T10:39:43.652248 /
2026-04-22T10:39:44.369304 /
```

Les données peuvent être conservées même après suppression du conteneur.

## 2.12 Inspection du volume

```
hodhod@hodhod-VMware-Virtual-Platform:~/devops-monitor$ sudo ls -la /var/lib/docker/volum
es/monitor-logs/_data/
[sudo] Mot de passe de hodhod :
total 12
drwxr-xr-x 2 hodhod hodhod 4096 avril 22 11:39 .
drwx----x 3 root   root   4096 avril 22 11:37 ..
-rw-r--r-- 1 hodhod hodhod 116 avril 22 11:39 visits.log
hodhod@hodhod-VMware-Virtual-Platform:~/devops-monitor$ sudo cat /var/lib/docker/volumes/
monitor-logs/_data/visits.log
2026-04-22T10:39:41.738285 /
2026-04-22T10:39:42.754818 /
2026-04-22T10:39:43.652248 /
2026-04-22T10:39:44.369304 /
```

Les données sont stockées physiquement sur la machine hôte.

## 2.13 Volume vs Bind mount

```
hodhod@hodhod-VMware-Virtual-Platform:~/devops-monitor$ mkdir -p ~/monitor-data
hodhod@hodhod-VMware-Virtual-Platform:~/devops-monitor$ docker run -d \
  --name monitor-bind \
  -p 8090:5000 \
  -v ~/monitor-data:/app/logs \
  devops-monitor:1.0
44c887bca93c9fffc1daad16b85184ac3a4a4314a2046ebe37f1159e09ea99cb7
hodhod@hodhod-VMware-Virtual-Platform:~/devops-monitor$ curl http://localhost:8090/

<html><body style='font-family:sans-serif;padding:40px'>
<h1>Bonjour depuis DevOps Monitor !</h1>
<p>Conteneur : <b>44c887bca93c</b></p>
<p>Visites : <b>1</b></p>
<p><a href='/health'>/health</a> · <a href='/stats'>/stats</a></p>
</body></html>
hodhod@hodhod-VMware-Virtual-Platform:~/devops-monitor$ cat ~/monitor-data/visits.log
2026-04-22T10:41:58.638281 /
```

Les volumes sont plus portables que les bind mounts.

## 2.14 Nettoyage final

```
hodhod@hodhod-VMware-Virtual-Platform:~/devops-monitor$ docker network rm app-network
app-network
hodhod@hodhod-VMware-Virtual-Platform:~/devops-monitor$ docker volume rm monitor-logs
monitor-logs
hodhod@hodhod-VMware-Virtual-Platform:~/devops-monitor$ docker system prune -a --volumes
WARNING! This will remove:
- all stopped containers
- all networks not used by at least one container
- all anonymous volumes not used by at least one container
- all images without at least one container associated to them
- all build cache

Are you sure you want to continue? [y/N] y
Deleted build cache objects:
b89k9fim4eefxtshkg9e0y4fq
un1e1b15dj9ghlci5q3qp7k89
a1envz4abvprkzvshc87xxwj7
1n0mi9drnae2mepm3t9ucmwj7
r456fmefbjzw7jtbdiklgmk7
sjukeu9iynfr6q4zzjp3ocmiy
s3xw47dd1i4u6kx6ya1ajpp30
ysdwahsvu33wy50y8jxf8wct3
xsbhjzz18njh82zh8l0jpzbrw
a9vuwtmmy3ff6rmtgxj180qkv
utnhihc4tabmilbd70q85q2rv
1ztksi6hwb6axd5mr5tehu83c
p0t203sevv6qkybn2ps1rcp4q

Total reclaimed space: 210.5MB
```

Docker permet de nettoyer rapidement tout l'environnement.

## 3 Conclusion

Ce TP a permis de comprendre les bases de Docker et son utilisation pour déployer des applications de manière simple et efficace.

Docker constitue une étape essentielle avant l'apprentissage de Kubernetes.